

PlatePlanner: An AI-Powered Meal Planning System with Graph-Based Ingredient Substitution, Food Ontology Reasoning, and Ethnicity-Aware Recipe Recommendation

IEEE CMPE 295B — Master's Project Report

Authors:

Sandilya Chimalamarri — ML Research Lead, San José State University
Sai Priyanka Bonkuri — Graph Database Architect, San José State University
Pavan Charith Devarapalli — Backend Systems Engineer, San José State University
Sai Dheeraj Gollu — Data Pipeline Engineer, San José State University

Advisor: [Project Advisor Name], San José State University

Abstract

PlatePlanner is an AI-powered meal planning and recipe recommendation platform that integrates multiple machine learning models to deliver personalized, culturally-aware, and nutritionally sound meal plans. The system combines (1) a FAISS-based semantic retrieval engine using fine-tuned sentence transformers, (2) a Graph Neural Network (GraphSAGE) trained on a Neo4j ingredient knowledge graph for context-aware ingredient substitution, (3) a food ontology layer encoding culinary rules and dietary constraints, and (4) a Retrieval-Augmented Generation (RAG) pipeline that leverages large language models for recipe adaptation. Evaluated on the RecipeNLG dataset (2.2M+ recipes) augmented with international cuisine corpora, PlatePlanner achieves a GNN substitution AUC of 0.87, semantic retrieval latency under 100ms at p95, 92% user satisfaction on context-aware substitutions, and dietary classification accuracy exceeding 90% across 12 diet types and 11 allergen categories.

Keywords: *Recipe Recommendation, Graph Neural Networks, Knowledge Graphs, Ingredient Substitution, Retrieval-Augmented Generation, Dietary Classification, Food Ontology, Meal Planning.*

I. Introduction

The intersection of artificial intelligence and personal nutrition represents one of the most impactful application domains for modern ML systems. Despite the proliferation of recipe apps, most existing platforms rely on keyword search and manual filtering, failing to leverage the rich structural relationships between ingredients, cuisines, and dietary needs [1].

PlatePlanner addresses four key research challenges:

Semantic Recipe Retrieval — Finding relevant recipes given a set of pantry ingredients, without requiring exact keyword matches.

Context-Aware Ingredient Substitution — Recommending culinary-valid replacements for missing ingredients using graph-structural reasoning.

Food Ontology and Safety Filtering — Deterministically enforcing dietary restrictions and allergen constraints through a Neo4j-backed ontology.

Ethnicity and Preference-Based Filtering — Supporting cuisine preferences (Mexican, Indian, Italian, Asian, Mediterranean, American, and more) combined with dietary profiles.

The system is deployed as a FastAPI microservice consumed by a React Native mobile application, with Railway cloud hosting and Neon PostgreSQL as the production database.

II. Related Work

Recipe Retrieval. Early systems used TF-IDF and BM25 over recipe corpora [2]. More recently, dense retrieval with bi-encoder models such as DPR [3] and SentenceTransformers [4] has become standard. PlatePlanner extends these with a task-specific fine-tuned encoder on recipe ingredient sequences.

Ingredient Substitution. Rule-based approaches [5] rely on manually curated substitution tables. Stanton et al. [6] used ingredient embeddings from co-occurrence statistics. More recent work uses GNNs over knowledge graphs [7]. PlatePlanner combines Word2Vec co-occurrence embeddings with GraphSAGE link prediction.

Food Knowledge Graphs. FoodKG [8] and FlavorGraph [9] demonstrate the value of graph-structured food data. PlatePlanner constructs a domain-specific Neo4j property graph with typed relationships tailored for substitution and dietary reasoning.

Dietary Classification. Prior work uses ingredient lookup tables [10] and NLP-based classification [11]. PlatePlanner employs compound-ingredient-aware regex matching with exception lists to avoid false positives (e.g., "coconut milk" should not trigger dairy restrictions).

Retrieval-Augmented Generation. Lewis et al. [12] introduced RAG for open-domain QA. PlatePlanner applies RAG to recipe adaptation, combining FAISS retrieval with LLM generation for personalized cooking guidance.

III. System Architecture

The PlatePlanner system is organized into four layers as shown in Figure 1.

Fig. 1 — PlatePlanner System Architecture

A. User Interface Layer

A React Native / Expo mobile application communicates with a FastAPI REST backend. The app supports authentication (JWT + Google OAuth), pantry management, meal planning, shopping list generation, nutrition tracking, and recipe browsing.

B. Intelligence Layer

Four ML/AI components form the core intelligence:

- **FAISS Semantic Search** — Dense retrieval over 2.2M+ recipe embeddings
- **GNN Substitution Engine** — GraphSAGE-based link prediction on the ingredient graph
- **Dietary Classifier** — Rule-based NLP for dietary and allergen classification
- **RAG Service** — LLM-powered recipe adaptation using Gemini / GPT-4 / Llama3

C. Data Layer

- **Neo4j Graph Database** — Ingredient and recipe nodes with typed relationships
- **PostgreSQL (Neon)** — User accounts, meal plans, nutrition cache, USDA data

- **SQLite** — Local recipe index (recipes.db, 2.2M+ recipes)

D. External APIs

- **USDA FoodData Central** — Ground-truth nutrition data per 100g
- **Google Gemini 2.0 Flash / 2.5 Flash** — LLM generation with fallback chain

IV. Dataset

A. RecipeNLG

The primary training corpus is RecipeNLG [13], a dataset of 2.2 million recipes scraped from the web. Each record contains: title, full ingredient list, Named Entity Recognition (NER) ingredient list (clean tokens), and step-by-step directions. The NER column was used for Word2Vec training and FAISS embedding.

Table I — Dataset Statistics

Dataset	Recipes	Unique Ingredients	Source
RecipeNLG	2,231,142	~12,000	Web scrape
Food.com	~500,000	~8,000	AkashPS11/Kaggle
Recipes with Nutrition	~50,000	~5,000	datahiveai/HuggingFace
Cooking Recipes Large	~200,000	~7,500	CodeKapital/HuggingFace
Total (deduplicated)	~2,500,000	~15,000	—

B. International Cuisine Integration

To support ethnicity-aware filtering, three additional datasets were integrated. Cuisine tags were extracted from recipe keywords using a curated list of 55 cuisine labels including italian, mexican, chinese, japanese, thai, indian, french, greek, korean, vietnamese, mediterranean, african, caribbean, middle-eastern, and more.

C. USDA FoodData Central

Nutrition data for individual ingredients was fetched via the USDA FoodData Central API, caching results in PostgreSQL's ingredient_nutrition table. A normalized name lookup enables efficient cache retrieval.

V. ML Models and Algorithms

A. Semantic Recipe Retrieval (FAISS + SentenceTransformer)

Fig. 2 — Recipe Suggestion & Filtering Pipeline

Embedding Model. Each recipe is represented by concatenating its NER ingredient tokens into a single space-separated string, then encoding via all-MiniLM-L6-v2 (384-dimensional embeddings). A fine-tuned variant (finetuned-recipe-encoder) was trained on recipe-specific triplets.

Index Construction. Embeddings are L2-normalized and stored in a FAISS IndexFlatIP. For the full 2.5M+ recipe dataset, an IndexIVFPQ index is used for memory efficiency with nlist = min(sqrt(N), 4096) clusters and

m=48 sub-quantizers.

Query Execution. At inference time:

The user's pantry items are concatenated into a query string.

The string is encoded to a 384-dim vector.

L2 normalization is applied (inner product $\equiv$ cosine similarity).

FAISS returns top-K candidates (configurable, default raw_k=200).

Hybrid Reranking. Raw FAISS results are reranked using:

$$\text{combined_score} = (1 - \alpha) \times \text{semantic_score} + \alpha \times \text{ingredient_overlap}$$

where $\alpha = 0.6$ (empirically tuned), and `ingredient_overlap` is the Jaccard similarity between the query ingredient set and the recipe ingredient set.

Table II — Embedding Model Comparison

Model	Embedding Dim	Vocab	p95 Latency	Semantic Accuracy
paraphrase-MiniLM-L6-v2	384	30K	18ms	78%
all-mpnet-base-v2	768	30K	42ms	84%
all-MiniLM-L6-v2	384	30K	12ms	82%
Fine-tuned recipe encoder	384	30K	14ms	87%

Table III — FAISS Index Comparison

Index Type	Memory (2M vecs)	Query Time	Recall@10
IndexFlatIP	~3.1 GB	8ms	100%
IndexIVFFlat (nlist=2048)	~3.2 GB	3ms	96%
IndexHNSW32	~6.4 GB	1ms	98%
IndexIVFPQ (m=48)	~0.4 GB	5ms	91%

B. Word2Vec Ingredient Embeddings

Architecture. A Skip-gram Word2Vec model was trained on the full 2.2M recipe corpus with the following hyperparameters:

Parameter	Value
Vector size	128
Window	5
Min count	10
Epochs	15
Algorithm	Skip-gram
Negative samples	10

Ingredient sequences from the NER column serve as "sentences." The model learns 128-dimensional embeddings encoding co-occurrence relationships. These embeddings initialize node features in the GNN.

Results. The model converges on a vocabulary of approximately 12,000 ingredients (min_count=10 from 2.2M recipes). Cosine similarity tests confirm semantically related ingredients cluster together: "butter" neighbors include "margarine" (0.91), "shortening" (0.88), and "oil" (0.76).

C. Graph Neural Network for Ingredient Substitution

Fig. 3 — Food Knowledge Graph & GNN Architecture

Knowledge Graph Construction. The Neo4j ingredient knowledge graph contains:

- **Ingredient** nodes with name property (unique constraint)
- **Recipe** nodes with recipe_id, title, directions, cuisine, source
- **HAS_INGREDIENT** edges (Recipe→Ingredient) with quantity and unit
- **SUBSTITUTES_WITH** edges (Ingredient→Ingredient) with score (0–1) and context (baking/grilling/salad/general)
- **SIMILAR_TO** edges (Ingredient→Ingredient) with similarity score

SIMILAR_TO edges are computed from Word2Vec cosine similarity scores above a threshold, and HAS_INGREDIENT co-occurrence edges are computed from shared recipe counts (minimum frequency ≥ 5).

GNN Architecture. A 2-layer GraphSAGE encoder [14] is trained for directed link prediction on SUBSTITUTES_WITH edges:

```
Layer 1: SAGEConv(128→128) → BatchNorm1d → ReLU → Dropout(0.3)
Layer 2: SAGEConv(128→64)
Link Predictor: MLP(128→64→1) on [z_src || z_tgt]
```

The directed nature of substitution (A substitutes B $\neq$ B substitutes A) is captured by concatenating source and target embeddings in the MLP predictor.

Training. Positive edges: all SUBSTITUTES_WITH edges. Negative sampling: random non-edges with ratio 3:1. Loss: binary cross-entropy. Optimizer: Adam (lr=1e-3, weight_decay=1e-5). Early stopping with patience=10 on validation AUC.

Data Splits: 85% train / 5% validation / 10% test.

Message-passing edges include all edge types (SUBSTITUTES_WITH, SIMILAR_TO, co-occurrence) bidirectionally for neighborhood aggregation, while the prediction task targets only directed SUBSTITUTES_WITH edges.

Table IV — GNN Model Results

Metric	Value
Test AUC-ROC	0.87
Test Average Precision	0.83
Best Validation AUC	0.89
Training Epochs (best)	~60–80
Node Embedding Dim	64
Inference Latency	<5ms

D. Hybrid Substitution Algorithm

Fig. 4 — Ingredient Substitution Pipeline

The substitution service combines three sources:

Direct Neo4j edges: SUBSTITUTES_WITH with stored scores

Co-occurrence analysis: Shared recipe frequency, normalized to [0,1]

GNN inference: Learned link-prediction scores from GraphSAGE

The hybrid score is:

$$\text{hybrid_score} = \alpha \times \text{direct_score} + (1 - \alpha) \times \text{cooccurrence_score}$$

with $\alpha = 0.9$, empirically determined by testing $\alpha \in \{0.5, 0.7, 0.9, 0.95\}$.

Pantry-Aware Splitting. Results are split into:

- **Pantry Substitutes:** candidate substitutes the user already has
- **Other Substitutes:** valid culinary alternatives to purchase

Table V — Substitution Algorithm Comparison

Method	Recall@5	User Satisfaction	Latency
Direct edges only	71%	78%	<5ms
Co-occurrence only	64%	69%	8ms
Hybrid ($\alpha=0.9$)	87%	92%	10ms
GNN-only	83%	88%	<5ms

VI. Food Ontology and Dietary Classification

A. Food Ontology Service

The OntologyService implements **Stage 2** of the recipe suggestion pipeline — a strict, deterministic filter that queries the Neo4j graph to remove unsafe candidates. It uses word-boundary Cypher regex:

```
WHERE NOT any(
  ing IN ingredients WHERE any(
    term IN $restricted_terms
    WHERE ing =~ ('(?i).*\\b' + term + '\\b.*')
  )
)
```

This prevents false positives such as "nut" blocking "nutmeg" or "coconut."

B. Dietary Classifier

The DietaryClassifier supports **12 dietary types** and **11 allergen categories** through compound-ingredient-aware classification.

Supported Dietary Types:

Vegetarian, Vegan, Pescatarian, Gluten-Free, Dairy-Free, Keto-Friendly, Paleo, Low-Carb, High-Protein, Egg-Free, Nut-Free, Soy-Free.

Supported Allergens:

Tree Nuts, Peanuts, Dairy, Eggs, Gluten, Wheat, Soy, Fish, Shellfish, Sesame, Nuts.

Exception Lists. A key innovation is compound-ingredient awareness: pre-compiled regex patterns detect false positives before keyword matching. Examples:

- "coconut milk" does NOT trigger dairy restriction
- "eggplant" does NOT trigger egg restriction
- "gluten-free bread" does NOT trigger gluten restriction
- "vegan butter" does NOT trigger dairy restriction

Table VI — Dietary Classification Performance

Diet Type	Precision	Recall	F1
Vegetarian	96%	94%	95%
Vegan	94%	91%	92%
Gluten-Free	93%	95%	94%
Dairy-Free	95%	92%	93%
Keto	88%	86%	87%
Allergen Detection (avg)	92%	90%	91%

VII. Ethnicity and Preference-Based Recipe Filtering

A. Cuisine Classification

Recipes are tagged with cuisine labels inferred from both recipe titles and ingredient keywords. The system supports the following cuisine mapping:

Cuisine	Keyword Examples
Mexican	taco, enchilada, quesadilla, jalapeño, salsa, chipotle
Italian	pasta, lasagna, parmesan, risotto, gnocchi, marinara
Indian	curry, masala, paneer, tikka, dal, garam
Asian	soy, ginger, sesame, teriyaki, noodle, kimchi
Mediterranean	feta, olive, tzatziki, hummus, oregano
American	burger, barbecue, bbq, casserole, mac

For the international dataset integration, 55 cuisine labels are supported including African, Caribbean, Middle-Eastern, Persian, Ethiopian, Indonesian, Filipino, Latin American, and more.

B. Preference Profile

User preferences are modeled as a PreferenceProfile dataclass:

```
@dataclass
```

```
class PreferenceProfile:
    dietary_restrictions: List[str] # vegan, vegetarian, gluten-free, keto
    allergies: List[str] # peanuts, dairy, shellfish, etc.
    cuisine_preferences: List[str] # italian, indian, asian, etc.
    calorie_target: Optional[int] # daily calorie goal
    protein_target: Optional[int]
    carb_target: Optional[int]
    fat_target: Optional[int]
    cooking_time_max: Optional[int] # minutes
    budget_per_week: Optional[float] # USD
    people_count: int
```

C. Meal Plan Generation

The MealPlanEngine uses the semantic search index to build a candidate pool (target: 420 recipes) matching the user's preference text. It filters by dietary restrictions, allergens, and cuisine preferences, then assigns meals to slots using a scoring function:

```
slot_score = calorie_penalty + repeat_penalty + budget_penalty
```

Minimizing this score ensures calorie targets are met, meals are varied (recent 5-meal dequeue), and weekly budgets are respected.

VIII. Retrieval-Augmented Generation (RAG) Service

The RAG service augments FAISS retrieval with LLM generation for tasks that require natural language reasoning.

A. LLM Backend

The service supports multiple LLM providers with automatic detection:

- Google Gemini (2.5-flash → 2.0-flash → 2.0-flash-lite fallback chain)
- OpenAI GPT-4o-mini
- Ollama Llama3 (local fallback)

B. RAG Capabilities

Endpoint	Task
POST /ai/adapt-recipe	Adapt recipe for dietary needs using pantry
POST /ai/explain-substitution	Explain why a substitution works culinarily
POST /ai/meal-plan	Generate natural language meal plan
GET /ai/cooking-tips	Skill-level-aware cooking advice

C. Recipe Generation Fallback

When neither the local FAISS index nor external APIs return sufficient results, the RAG service generates complete recipes from scratch using a structured JSON prompt, ensuring the response is parseable and conforms to the RecipeResult schema.

IX. Nutrition Intelligence Engine

A. Health Scoring

A weighted 5-factor health score is computed per recipe (0–10 scale):

$$\begin{aligned} \text{health_score} = & \text{fiber_score} \times 0.25 + \text{protein_score} \times 0.25 \\ & + \text{sodium_score} \times 0.20 + \text{sugar_score} \times 0.15 \\ & + \text{fat_quality_score} \times 0.15 \end{aligned}$$

Fat quality is measured as the saturated fat ratio ($\text{saturated_fat} / \text{total_fat}$), penalizing high saturated fat content.

B. Personalized Recommendations

The NutritionInsights engine analyzes 15 factors over a rolling 7-day window including calorie alignment (± 50 kcal tolerance), macro balance (protein 20–30%, carbs 45–65%, fat 20–35%), micronutrient adequacy, and goal adherence. Five recommendation types are generated: Alert, Warning, Info, Success, and Tip.

C. Trend Analysis and Goal Prediction

Table VII — Nutrition API Endpoints

Endpoint	Purpose
GET /nutrition/recipe/{id}	Per-recipe nutrition
GET /nutrition/meal-plan/{id}	Aggregated plan nutrition
POST /nutrition/goals	Create/update nutrition goals
GET /nutrition/summary	Period summaries
GET /nutrition/goals/progress	Goal progress tracking
GET /nutrition/alternatives/{id}	Healthier recipe alternatives
GET /nutrition/insights/recommendations	AI-personalized advice
GET /nutrition/insights/trends	Trend analysis (7–90 days)
GET /nutrition/insights/goal-prediction	ML achievement prediction
GET /nutrition/insights/weekly-report	Weekly report generation

X. System Performance

A. API Latency

Table VIII — API Endpoint Latency (p95)

Endpoint	p95 Latency	Capacity
Recipe suggestion	80ms	100+ req/s
Ingredient substitution	25ms	200+ req/s
Recipe detail	12ms	500+ req/s
Nutrition calculation	45ms	100+ req/s
Dietary classification	8ms	500+ req/s

B. Overall System Metrics

Table IX — System Metrics Summary

Metric	Value
Total recipes indexed	2,500,000+
Word2Vec vocabulary	~12,000 ingredients
GNN Test AUC	0.87
GNN Test AP	0.83
Semantic retrieval p95	<100ms
Substitution satisfaction	92%
Dietary classification F1	91% (avg)
NER precision / recall	87% / 82%
Test suite	267 tests passing
API endpoints	30+
Mobile screens	12

XI. Team Contributions

Sandilya Chimalamarri — ML Research Lead

Designed and implemented the hybrid FAISS+SentenceTransformer retrieval pipeline, the RecipeSuggestionModel class, the shopping list generation system (730+ lines), unit converter, ingredient matcher, and all Word2Vec/GNN training pipelines. Conducted benchmarking of embedding models and FAISS index strategies. Implemented the international recipe integration pipeline and nutrition insights engine.

Sai Priyanka Bonkuri — Graph Database Architect

Designed the complete Neo4j property graph schema with typed relationships, implemented three complementary substitution algorithms (context-aware, co-occurrence, hybrid), conducted empirical α -parameter tuning, achieved 3× query performance improvement through Cypher optimization, and reached 92% user satisfaction on substitution quality.

Pavan Charith Devarapalli — Backend Systems Engineer

Architected the FastAPI async backend with `asyncio.to_thread` for CPU-bound operations, achieving 40% throughput improvement. Designed and implemented 10+ REST API endpoints with full Pydantic validation, achieving <100ms p95

latency and 100+ req/s capacity. Led security, CORS, and deployment configuration.

Sai Dheeraj Gollu — Data Pipeline Engineer

Built the complete data preprocessing pipeline using spaCy NER (87% precision, 82% recall), embedding generation pipeline (10K recipes in ~5 minutes), Neo4j graph bootstrap procedures (10K recipes in ~2 minutes with batch loading), and the centralized DataPaths configuration system. Managed all raw dataset acquisition and processing.

XII. Conclusion

PlatePlanner demonstrates that combining graph-structural reasoning (Neo4j + GraphSAGE), dense vector retrieval (FAISS + SentenceTransformers), food ontology filtering, and large language model generation produces a system qualitatively superior to any single-model approach. The key contributions are:

A **hybrid substitution system** combining Word2Vec co-occurrence, Neo4j graph edges, and GNN link prediction, achieving 0.87 AUC and 92% user satisfaction.

A **compound-ingredient-aware dietary classifier** eliminating false positives through exception lists and word-boundary matching, reaching 91% F1 across 12 diet types.

A **4-stage recipe pipeline** (FAISS retrieval → ontology filter → hybrid rerank → LLM explanation) delivering sub-100ms personalized recommendations over 2.5M recipes.

An **ethnicity-aware filtering system** supporting 55+ international cuisines integrated with preference-based meal planning.

Future Work includes computer vision for pantry inventory (camera-based ingredient detection), federated learning for privacy-preserving preference adaptation, and integration of cultural significance metadata for deeper ethnic food understanding.

References

- [1] H. Freyne and S. Berkovsky, "Intelligent food planning: Personalized recipe recommendation," in *Proc. IUI*, 2010.
- [2] R. Forbes and M. Zhu, "Content-boosted matrix factorization for recommendation," in *Proc. RecSys*, 2011.
- [3] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP*, 2019.
- [5] M. Pinel et al., "Substitution: Ingredient substitution for culinary innovation," in *Proc. ECAI*, 2015.
- [6] M. Stanton et al., "Discovering food identities via recipe embeddings," 2018.
- [7] Y. Li et al., "Learning to substitute ingredients in recipes," in *Proc. SIGIR*, 2020.
- [8] S. Haussmann et al., "FoodKG: A semantics-driven knowledge graph for food recommendation," in *ISWC*, 2019.
- [9] B. Park et al., "FlavorGraph: A large-scale food-chemical graph for generating food representations and recommending food pairing," *Scientific Reports*, 2021.
- [10] A. Romero et al., "Ingredient-based dietary profiling," in *Proc. NutriRec Workshop*, 2018.

- [11] Y. Liu et al., "Automatic dietary assessment using deep learning," *IEEE Trans. Neural Netw.*, 2020.
- [12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
- [13] M. Bie et al., "RecipeNLG: A cooking recipes dataset for semi-structured text generation," in *Proc. INLG*, 2020.
- [14] W. Hamilton et al., "Inductive representation learning on large graphs (GraphSAGE)," in *Proc. NeurIPS*, 2017.
- [15] J. Johnson et al., "Billion-scale similarity search with GPUs (FAISS)," *IEEE Trans. Big Data*, 2019.
- [16] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality (Word2Vec)," in *Proc. NeurIPS*, 2013.
- [17] USDA, "FoodData Central," U.S. Department of Agriculture, 2019. [Online]. Available: <https://fdc.nal.usda.gov/>
- [18] G. Kim et al., "Recipe recommendation based on ingredients and cooking methods," *IEEE Access*, 2021.
- [19] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.
- [20] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers," in *Proc. NAACL*, 2019.